

فاز ۸: پایگاه داده (Database) – کامل و عمیق

این فاز به دو بخش بزرگ تقسیم میشه:

• بخش اول: اصول طراحی پایگاه داده (Database Design)

• بخش دوم: آژ پایگاه داده (SQL)

بخش اول: اصول طراحی پایگاه داده (Database Design)

هدف این بخش اینه که یاد بگیریم چطور به مسئله دنیای واقعی رو به یه مدل داده‌ای بهینه و بدون افزونگی (Redundancy) تبدیل کنی. مصاحبه‌کننده می‌خواد ببینه تو فقط SQL بلد نیستی، بلکه می‌تونی به دیتابیس رو از صفر طراحی کنی.

8.1.1 مدل‌سازی داده (Data Modeling)

۱. مدل (ER (Entity-Relationship Model):

• موجودیت (Entity): یه شیء یا مفهوم در دنیای واقعی که می‌خوایم اطلاعاتش رو ذخیره کنیم. (مثلاً Customer, Order, Product). در نهایت به جدول تبدیل میشه.

• صفت (Attribute): ویژگی‌های یه موجودیت. (مثلاً CustomerName, OrderDate). به ستون تبدیل میشن.

• رابطه (Relationship): ارتباط بین دو موجودیت.

• One-to-One (1:1): یه User یک Profile داره.

• One-to-Many (1:N): یه Customer می‌تونه چندین Order داشته باشه. (پرکاربردترین).

• Many-to-Many (M:N): یه Student چندین Course داره، و یه Course چندین Student داره. (نیاز به جدول واسطه داره).

کلیدها (Keys):

• Primary Key (PK - کلید اصلی): ستونی که به طور یکتا (Unique) هر سطر رو مشخص می‌کنه. (مثلاً CustomerID). نمی‌تونه NULL باشه. هر جدول فقط یک PK داره.

• Foreign Key (FK - کلید خارجی): ستونی در جدول فرزند که به کلید اصلی جدول والد اشاره می‌کنه. (مثلاً در جدول Order، ستون CustomerID به FK هست که به Customer.CustomerID اشاره می‌کنه). برای حفظ یکپارچگی ارجاعی (Referential Integrity) به کار میره.

8.1.2 نرمال‌سازی (Normalization)

? سوال مصاحبه‌ای ۱: نرمال‌سازی چیه و چرا انجامش میدیم؟ (سوال تکراری در همه مصاحبه‌ها)

جواب: فرآیند سازماندهی ستون‌ها و جداول برای کاهش افزونگی (Redundancy) و بهبود یکپارچگی (Integrity) داده‌ها. هدف اصلی اینه که هر داده فقط و فقط در یک جا ذخیره بشه. این کار از بروز ناهنجاری‌ها (Anomalies) در عملیات INSERT, UPDATE, DELETE جلوگیری می‌کنه.

سه شکل نرمال اول (3NF) رو باید مثل آب خوردن بلد باشی:

1NF (First Normal Form)

• قانون: هر خانه از جدول باید تک‌مقداری (Atomic) باشه. یعنی نمی‌تونی توی یه سلول، چند تا شماره تلفن بذاری که با کاما جدا شدن.

• همچنین: هر سطر باید یکتا باشه (پس حتماً یه کلید اصلی نیاز داریم).

غلط (نقض 1NF)	درست (1NF)
StudentID, Courses	StudentID, CourseID
Math, Physics ,1	Math ,1
	Physics ,1

:2NF (Second Normal Form)

•پیش‌نیاز: جدول باید در 1NF باشد.

•قانون: هیچ صفت غیرکلیدی (Non-prime attribute) نباید فقط به بخشی از کلید اصلی وابسته باشد. (این موضوع فقط وقتی مطرح میشه که کلید اصلی ما ترکیبی (Composite Key) باشه).

•مثال: جدولی با کلید ترکیبی (EmployeeID, SkillID). اگه ستون EmployeeName توی این جدول باشه، این فقط به EmployeeID وابسته است، نه به کل کلید. پس باید این جدول بشکنه و EmployeeName به جدول Employee بره.

:3NF (Third Normal Form)

•پیش‌نیاز: جدول باید در 2NF باشد.

•قانون: هیچ صفت غیرکلیدی نباید به یک صفت غیرکلیدی دیگه وابسته باشه (بیش می‌گن وابستگی گذرا یا Transitive Dependency). یعنی همه ستون‌های غیرکلیدی باید فقط و فقط به کلید اصلی وابسته باشن.

•مثال کلاسیک: جدول Employee: EmployeeID (PK), DepartmentID (FK), DepartmentName

•اینجا DepartmentName به DepartmentID وابسته است، نه به EmployeeID. پس باید DepartmentName رو ببریم توی یه جدول Department.

8.1.3 ایندکس‌ها (Indexes)

? سوال مصاحبه‌ای ۲: ایندکس چیه؟ مزایا و معایبش چیه؟ کی باید ایندکس بسازیم؟

جواب: ایندکس یه ساختمان داده (معمولاً B-Tree) هست که روی یک یا چند ستون از جدول ساخته میشه تا سرعت جستجو و واکنش داده (SELECT با شرط WHERE) رو به شدت افزایش بده. مثل فهرست انتهای کتاب می‌مونه.

•مزیت: خواندن (SELECT) رو سریع‌تر می‌کنه.

•معایب:

•نوشتن (INSERT, UPDATE, DELETE) رو کندتر می‌کنه، چون خود ایندکس هم باید به‌روز بشه.

•فضای اضافی روی دیسک اشغال می‌کنه.

•کی ایندکس بسازیم؟ روی ستون‌هایی که زیاد توی ORDER BY، JOIN، WHERE و GROUP BY استفاده میشن.

•ستون‌های کلید خارجی (FK) کاندیدای عالی برای ایندکس هستن.

بخش دوم: آژ پایگاه داده (SQL)

اینجا دیگه عملاته. باید بتونی کوئری بنویسی، Join بزنی، و داده‌ها رو گروه‌بندی کنی.

8.2.1 عملیات CRUD

•SELECT: خواندن داده.

•INSERT: اضافه کردن داده.

•UPDATE: به‌روزرسانی داده.

•DELETE: حذف داده.

8.2.2 JOINها (حیاتی)

? سوال مصاحبه‌ای ۳: انواع JOIN رو با مثال توضیح بده

•جواب: فرض کن دو جدول داریم: Customers (مشتریان) و Orders (سفارشات).

•INNER JOIN (یا JOIN): فقط سطرهایی که در هر دو جدول تطابق دارن رو برمی‌گردونه.

```

1 SELECT * FROM Customers
2 INNER JOIN Orders ON Customers.CustID = Orders.CustID;
3 -- فقط مشتریانی که سفارش دارند

```

LEFT JOIN (یا LEFT OUTER JOIN): همه سطرهای جدول چپ (Customers) رو برمی‌گردونه. اگه جدول راست تطابقی نداشته، ستون‌های راست NULL میشن

SELECT * FROM Customers LEFT JOIN Orders ON Customers.CustID = Orders.CustID; -- همه مشتریان. آنهایی که سفارش ندارند، اطلاعاتشون کنارشون NULL هست.

RIGHT JOIN (یا RIGHT OUTER JOIN): عکس LEFT JOIN. همه سطرهای جدول راست. (کمتر استفاده میشه، میتونی با LEFT JOIN و جابجا کردن جدول‌ها همون کار رو بکنی)

8.2.3 گروه‌بندی و توابع تجمعی

؟ سوال مصاحبه‌ای ۴: GROUP BY و HAVING چه فرقی دارن؟

• GROUP BY: سطرها رو بر اساس یک یا چند ستون گروه‌بندی می‌کنه. معمولاً با توابع تجمعی (COUNT, SUM,)

(AVG, MAX, MIN) استفاده میشه تا به خلاصه از هر گروه به دست بیاد.

• WHERE در مقابل HAVING:

•WHERE: سطرها رو قبل از گروه‌بندی فیلتر می‌کنه. نمی‌تونه روی توابع تجمعی شرط بذاره.

•HAVING: گروه‌ها رو بعد از گروه‌بندی فیلتر می‌کنه. برای شرط گذاشتن روی توابع تجمعی طراحی شده

(مثلاً بگو فقط گروه‌هایی رو نشون بده که تعدادشون بیشتر از ۵ تاست).

1 -- a. مثال: تعداد سفارشات هر مشتری، فقط برای مشتریانی که بیش از ۲ سفارش دارن

2 SELECT CustomerID, COUNT(*) AS OrderCount

3 FROM Orders

4 GROUP BY CustomerID

5 HAVING COUNT(*) > 2;

8.2.4 زیرپرس‌وجوها (Subqueries)

? سوال مصاحبه‌ای ۵: Subquery چیه و کجا استفاده میشه؟

جواب: یه کوئری که داخل یه کوئری دیگه قرار میگیره. معمولاً داخل WHERE، FROM یا SELECT.

1 -- a. پیدا کردن مشتریانی که سفارش‌هاشون از میانگین کل بیشتره

2 SELECT * FROM Orders

3 WHERE Amount > (SELECT AVG(Amount) FROM Orders);

4

5 ## ترجیح داده میشه JOIN هست و JOIN معمولاً کندتر از WHERE در Subquery: نکته

8.2.5 مفاهیم پیشرفته SQL

•UNION و UNION ALL:

•این ستون‌ها رو روی هم می‌چسبونن (نه مثل JOIN که کنار هم می‌چسبونن). یعنی نتیجه دو کوئری رو

باهم ترکیب می‌کنن. تعداد و نوع ستون‌ها باید یکسان باشه.

• UNION ALL: همه سطرها را نگه میدارد (حتی تکراری). سریعتر.

• UNION: سطرهای تکراری را حذف می‌کند (کندتر چون باید مرتب‌سازی و حذف انجام بده).

• EXISTS vs IN

• IN: بررسی می‌کند که به مقدار درون یه لیست یا نتیجه Subquery هست یا نه. برای لیست‌های کوچک خوبه.

• EXISTS: بررسی می‌کند که آیا Subquery حداقل یک سطر برمی‌گردونه یا نه. معمولاً برای Subquery های بزرگ سریعتر از IN هست، چون به محض پیدا کردن اولین سطر تطابق، متوقف میشه.

اینم از فاز ۸. همونطور که می‌بینی، خیلی وسیع و مهمه. ولی نگران نباش، با تمرین SQL توی LeetCode یا HackerRank کاملاً مسلط میشی.

؟ سوال مصاحبه‌ای ۶: تفاوت SQL و NoSQL چیه؟ کی از کدوم استفاده کنیم؟

جواب: این یکی از مهم‌ترین سوالات مصاحبه‌ست.

ویژگی	SQL (Relational)	NoSQL (Non-Relational)
ساختار	جدولی (Table) با Schema ثابت	مستند (Document)، کلید-مقدار، گراف
زبان	SQL (استاندارد)	API اختصاصی هر دیتابیس
تراکنش	ACID (قوی)	BASE (انعطاف‌پذیر)
مقیاس‌پذیری	عمودی (Vertical)	افقی (Horizontal)
نمونه‌ها	MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra

کی از SQL استفاده کنیم؟

• داده‌ها ساختار مشخص دارن

• نیاز به Join و Query پیچیده داریم

• یکپارچگی داده مهمه (مثل بانک، حسابداری)

کی از NoSQL استفاده کنیم؟

• داده‌ها ساختار مشخصی ندارند (مثل JSON)

• سرعت و مقیاس‌پذیری مهمه

• داده‌ها به راحتی Shard میشن (مثل سیستم‌های بزرگ)

? سوال مصاحبه‌ای ۷: Redis چیه و چه کاربردی داره؟

جواب: Redis یک دیتابیس In-Memory (در حافظه) از نوع Key-Value هست. خیلی سریع‌ه چون داده‌ها رو در RAM نگه می‌داره.

کاربردهای اصلی:

1. Cache (کش): ذخیره نتایج پرتکرار برای سرعت بخشیدن

2. Session Storage: ذخیره اطلاعات نشست کاربر

3. Rate Limiting: محدود کردن تعداد درخواست‌ها

4. Message Queue: صف پیام‌های ساده (با Pub/Sub)

5. Counter: شمارنده‌های سریع (مثل تعداد بازدید)

Redis دستورات ساده # 1

2 SET user:1 "Ali" # ذخیره

3 GET user:1 # دریافت "Ali" ->

4 INCR page_views # افزایش شمارنده

5 EXPIRE session:1 3600 # منقضی شدن بعد از ۱ ساعت

مقایسه با دیتابیس معمولی:

Redis: فوق العاده سریع، ولی داده‌ها موقتی هستند و گروانه

دیتابیس معمولی: کندتر، ولی دائمی و ارزون

? سوال مصاحبه‌ای ۸: Transaction Isolation Level ها رو توضیح بده

جواب: وقتی چندین تراکنش همزمان اجرا میشن، ممکنه مشکلاتی پیش بیاد. ۴ سطح ایزوله‌سازی وجود داره:

سطح ایزولاسیون	Dirty Read	Non-Repeatable Read	Phantom Read
READ_UNCOMMITTED	ممکنه ❌	ممکنه ❌	ممکنه ❌
READ_COMMITTED	غیرممکن ✅	ممکنه ❌	ممکنه ❌
REPEATABLE_READ	غیرممکن ✅	غیرممکن ✅	ممکنه ❌
SERIALIZABLE	غیرممکن ✅	غیرممکن ✅	غیرممکن ✅

توضیح مشکلات:

Dirty Read: خواندن داده‌ای که هنوز Commit نشده

Non-Repeatable Read: دو بار خواندن یه ردیف، مقادیر متفاوت

Phantom Read: یه Query دو بار اجرا میشه و تعداد ردیف‌ها فرق داره

1 SQL تنظیم سطح ایزوله‌سازی در --

2 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;